Universidad El Bosque

Ingeniería de Sistemas

Bases de Datos 2

Taller Integrador — MercaAndes

Documento de respuestas

Mariana Camila Durán Piñeros

David Santiago Gómez Durán

Parte 1. Transacciones, concurrencia y bloqueos

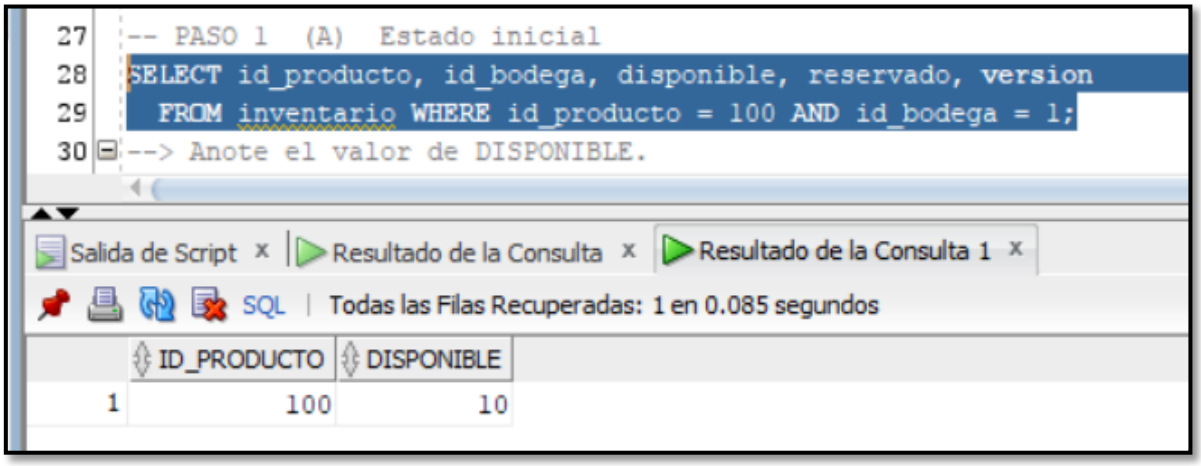
a. Provoque una actualización perdida sobre el inventario del producto 100 y explique por qué el bloqueo de fila no la impidió.

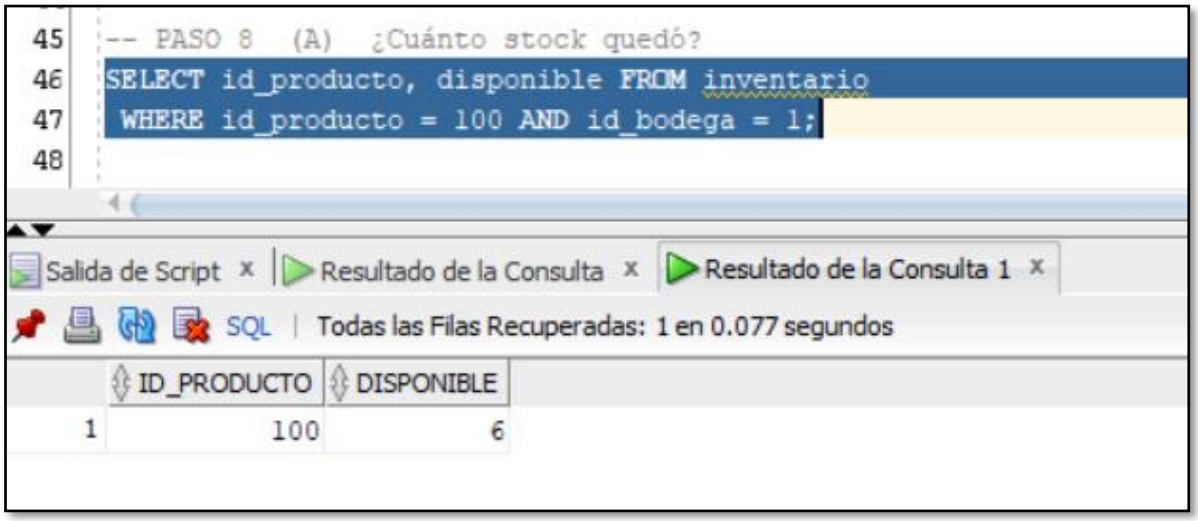
Empezamos con 10 unidades del producto 100. La sesión A vendió 3 y la sesión B vendió 4, o sea que en total se vendieron 7 y deberían haber quedado 3. Pero quedaron 6. Ninguna de las dos sesiones mostró un error: las dos dijeron que habían actualizado una fila y las dos guardaron sin problema. La venta de A simplemente desapareció.

El bloqueo no falló, cuando B intentó guardar su cambio, se quedó esperando a que A terminara, así que las dos escrituras se hicieron una después de la otra y nunca al mismo tiempo. Oracle tampoco dejó que B alcanzara a ver el 7 que A había escrito, pero todavía no guardaba: en el paso 3, B vio 10.

El problema estuvo en que la aplicación hizo la resta por su cuenta, con un dato desactualizado. B había leído 10, calculó 10 − 4 = 6, y cuando le dieron el turno, escribió ese 6 encima del 7 que A ya había dejado. El UPDATE pusó un número fijo (SET disponible = 6), y ese número venía de una lectura vieja. En resumen, el bloqueo cuida la fila en el momento de escribir, pero no puede hacer nada con una cuenta que se hizo antes.

Paso 1 y Paso 8





b. Corrija el mismo escenario con bloqueo pesimista (SELECT ... FOR UPDATE) y con bloqueo optimista (columna de versión). Compare ambas estrategias.

Pesimista: Con SELECT ... FOR UPDATE quedaron las 3 unidades correctas. Cambiaron dos cosas respecto al escenario anterior: la fila queda bloqueada desde que se lee y no desde que se escribe, y el UPDATE pasó a ser una resta (disponible = disponible - 4), o sea que el valor nuevo lo calcula el motor sobre el dato vigente.

Optimista: Con la columna de versión nadie bloqueó nada. B escribió primero y la versión pasó de 0 a 1, así que cuando A intentó escribir con version = 0 su UPDATE actualizó 0 filas. Ahí se detectó el choque. Ante ese resultado la aplicación debe releer la fila y reintentar; lo que nunca puede hacer es ignorarlo, porque cero filas actualizadas no es un error para Oracle y el fallo pasaría en silencio.

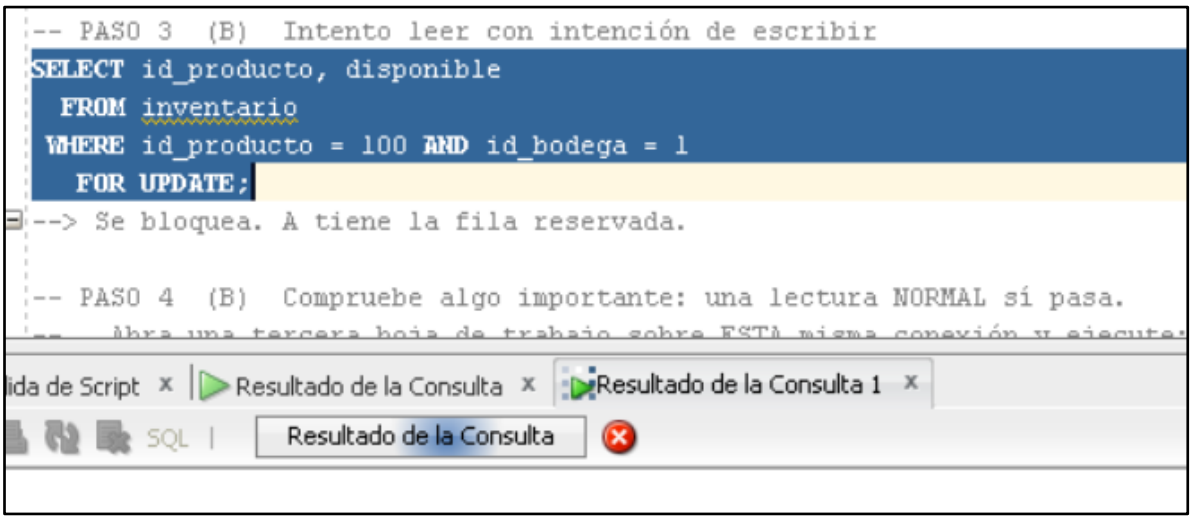
Comparación: Las dos evitan la pérdida, pero por caminos opuestos. La pesimista bloquea la fila desde que la lee, así que el choque nunca llega a ocurrir. La optimista deja que las dos sesiones avancen a la vez, el choque sí ocurre, y se detecta al final cuando el UPDATE devuelve cero filas.

Eso cambia lo que le pasa al que pierde la carrera. Con la pesimista, B esperó su turno, pero completó su venta. Con la optimista, A no esperó nada, pero su operación no se aplicó y tendría que volver a empezar.

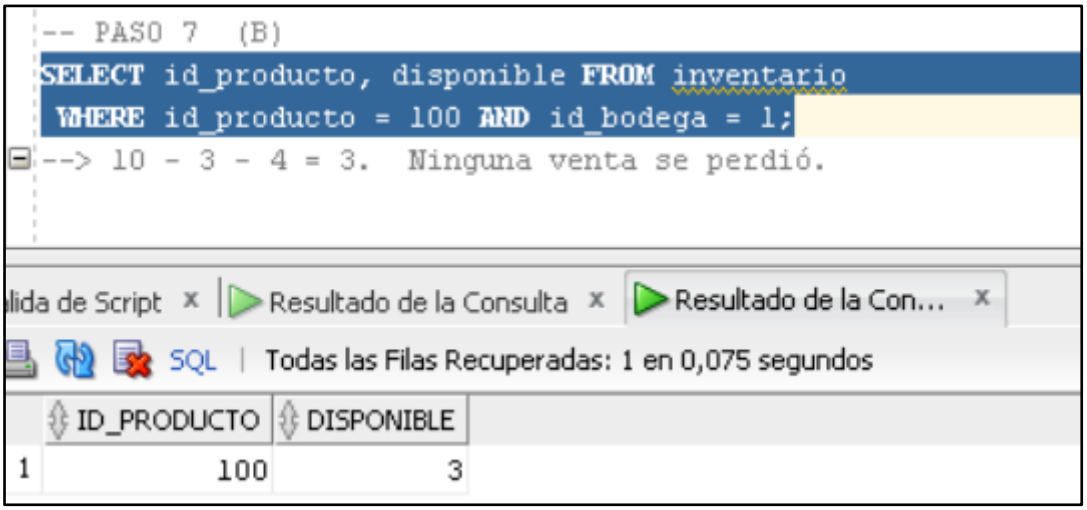
También cambia quién responde por que funcione. Con la pesimista lo garantiza la base de datos sola. Con la optimista queda en manos de la aplicación, porque cero filas actualizadas no es un error y si nadie revisa ese número el fallo pasa en silencio.

Cuál conviene depende de qué tan seguido choquen dos sesiones por la misma fila. En INVENTARIO, que todos los pedidos tocan y donde con solo 5 bodegas el tráfico cae sobre pocas filas, conviene la pesimista. En CLIENTE o PEDIDO, con 20.000 y 200.000 filas que casi siempre toca una sola persona, conviene la optimista.

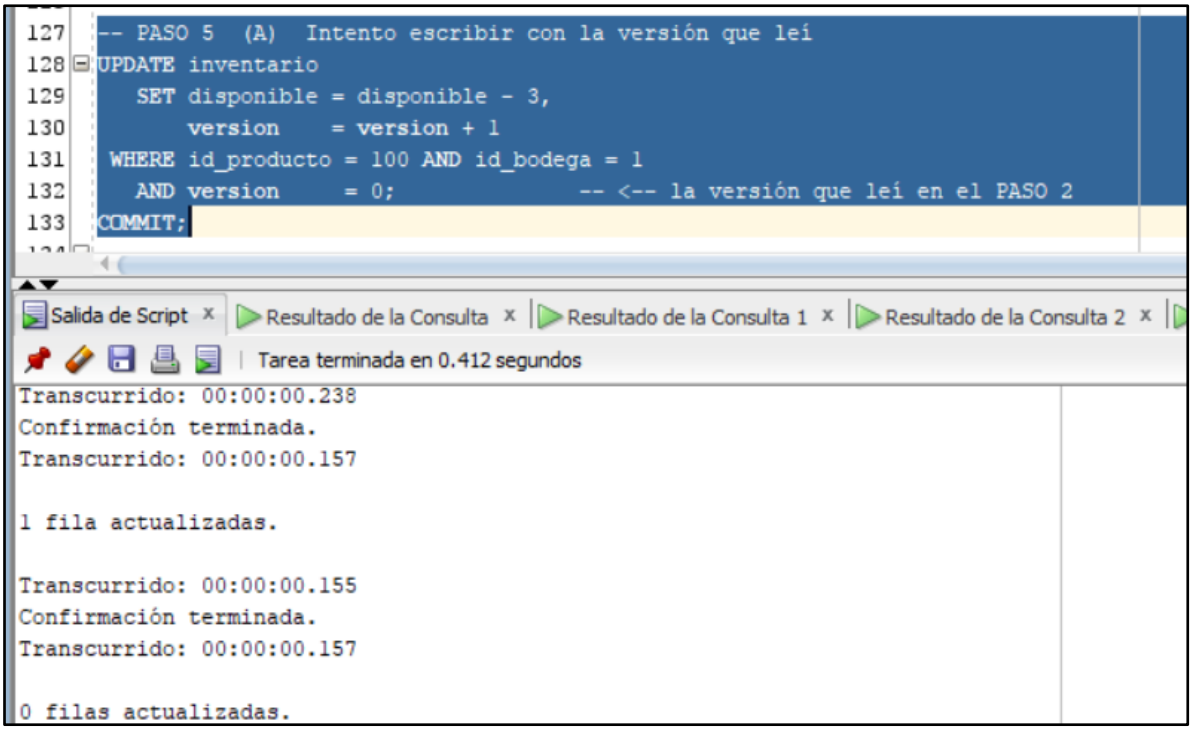
Paso 3 del ejercicio 1.2: se queda esperando



Paso 7 del ejercicio 1.2: no se perdió ninguna venta



Paso 5 del ejercicio 1.3: como hay choque, no deja actualizar filas (optimista)

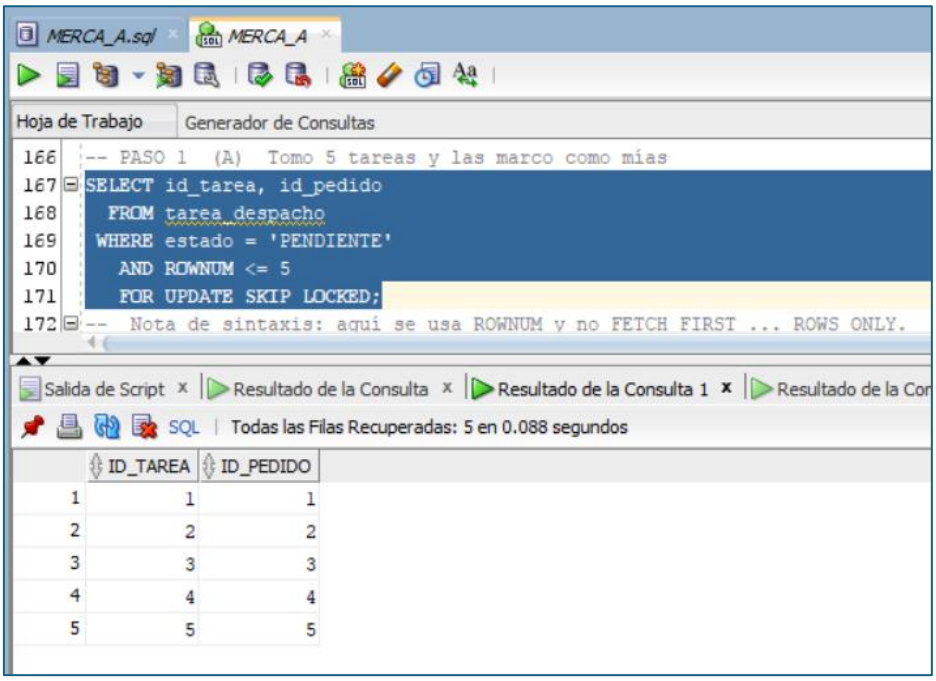


c. Reparta la cola de despacho entre las dos sesiones usando FOR UPDATE SKIP LOCKED y verifique que no haya tareas repetidas.

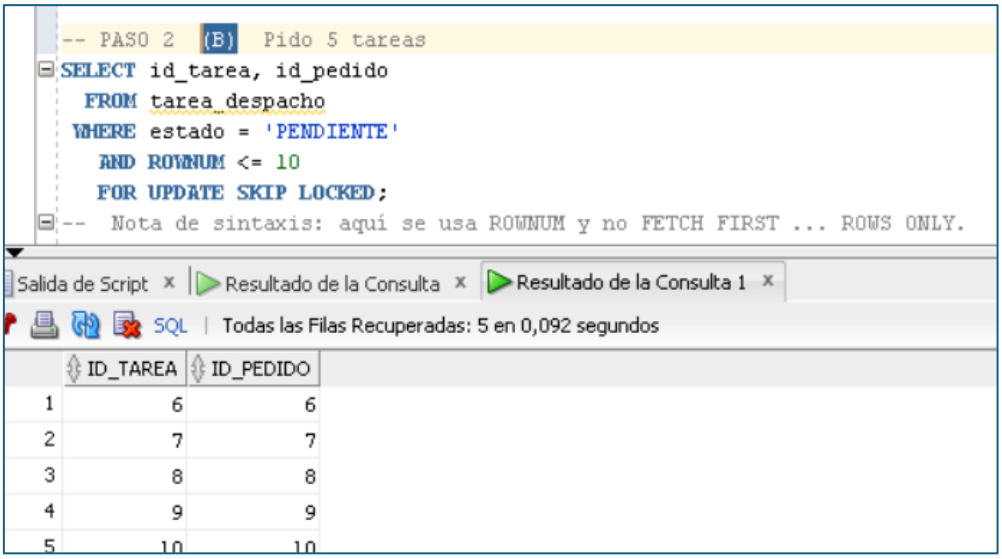
A tomó 5 tareas y no confirmó. B pidió 5 y recibió otras 5 distintas, sin esperar nada. Al comparar, no hubo ninguna tarea repetida.

Sin SKIP LOCKED habría pasado una de dos cosas. Con FOR UPDATE a secas, B se queda esperando indefinidamente y la cola se vuelve secuencial, así que dos operarios rinden lo mismo que uno. Con FOR UPDATE NOWAIT, B falla de una con ORA-00054: resource busy, sin esperar, pero también sin trabajar.

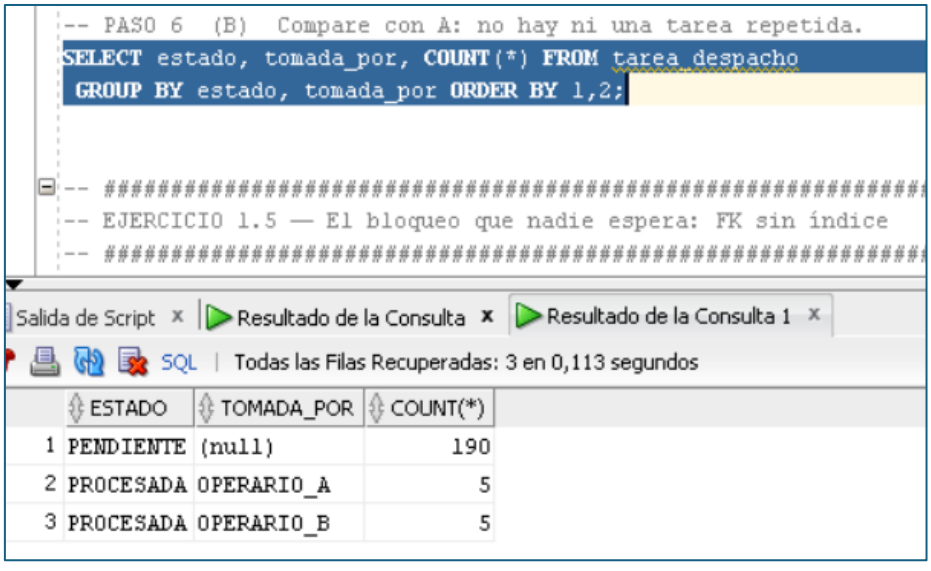
Paso 1: a la sesión A se le asignan 5 tareas



Paso 2: a la sesión B se le asignan 5 tareas diferentes



Paso 6: cantidad de tareas de cada uno



d. Reproduzca el bloqueo de tabla que provoca una llave foránea sin índice, corríjalo y repita la prueba.

A insertó un pago del pedido 900001 y no confirmó. B intentó borrar el pedido 900005, que no tiene ningún pago ni detalle, y quedó bloqueada. Las dos sesiones no compartían una sola fila. En la consulta de v$lock aparece un bloqueo de tipo TM, que es de tabla, y no solo el TX de fila.

Este fenómeno se explica porque PEDIDO es la tabla padre y PAGO la hija: un pago no puede existir sin su pedido. Al borrar un pedido, Oracle tiene que comprobar que no queden pagos apuntando a él. Como PAGO.ID_PEDIDO no tiene índice, esa comprobación obliga a recorrer la tabla de pagos entera, y mientras la recorre otra sesión podría insertar un pago del pedido que se está borrando. Para evitarlo, Oracle bloquea toda la tabla PAGO durante la operación. Ese bloqueo choca con el INSERT sin confirmar de A.

Tras crear ix_pago_pedido y repetir la prueba, el DELETE respondió de inmediato: la comprobación pasó a ser una búsqueda puntual y bastó con bloqueos de fila.

Paso 3 del ejercicio 1.5: se evidencia que la tabla PAGO tiene un bloqueo a nivel de tabla

```
220  -- PASO 3  (A)  Mientras B espera, mire el bloqueo.
221  --   TX = bloqueo de fila.  TM = bloqueo de TABLA.
222  SELECT l.sid, l.type,
223         DECODE(l.lmode, 0,'ninguno',1,'null',2,'RS',3,'RX',4,'S',5,'SRX',6,'X') AS tiene,
224         DECODE(l.request,0,'-',1,'null',2,'RS',3,'RX',4,'S',5,'SRX',6,'X')      AS pide
225    FROM v$lock l JOIN v$session s ON s.sid = l.sid
226   WHERE s.username = USER AND l.type IN ('TX','TM');
227  --> Si no tiene permisos sobre V$, el docente lo proyecta.
228
229  -- PASO 4  (A)  Libero
230  COMMIT;
```

Salida de Script | Resultado de la Consulta | Resultado de la Consulta 1 | Resultado de la Consulta 2 | Result

SQL | Todas las Filas Recuperadas: 8 en 0.602 segundos

	SID	TYPE	TIENE	PIDE
1	43657	TX	X	-
2	17625	TM	RX	-
3	43657	TM	RX	-
4	17625	TX	X	-
5	43657	TX	ninguno	S
6	43657	TM	RS	-
7	17625	TM	RX	-
8	43657	TM	RX	-

Después de crear el índice, la fila se borra de inmediato

```
1 fila eliminado

Transcurrido: 00:00:00.158
```

e. Provoque un interbloqueo (ORA-00060) y formule la regla de diseño que lo habría evitado.

En este punto, A reservó el producto 100 y B el 200. Luego, A pidió el 200 y se quedó esperando; B pidió el 100 y se cerró el círculo. A los pocos segundos, la sesión A recibió ORA-00060: deadlock detected while waiting for resource.

Oracle canceló solo la sentencia que cerró el círculo, no toda la transacción, así que la sesión víctima quedó con sus bloqueos puestos y estorbando a las demás. Tuvo que hacer ROLLBACK y empezar la operación otra vez desde el principio.

Regla de diseño: toda transacción que necesite bloquear varias filas debe tomarlas siempre en el mismo orden, por ejemplo, id_producto ascendente. Si las dos sesiones piden primero el 100 y después el 200, la segunda simplemente espera. El círculo solo se arma cuando una va 100→200 y la otra 200→100.

Parte 2. Índices y optimización

a. Explique por qué el informe mensual hace un recorrido completo de la tabla PEDIDO y corríjalo con un índice compuesto. Justifique el orden de las columnas.

	Operación	COST	Filas
Antes del índice	TABLE ACCESS FULL	309	5.289
Después del índice	TABLE ACCESS FULL	309	4.715

En un primer momento, PEDIDO solo tenía el índice de su llave primaria, sobre id_pedido, y la consulta no filtra por esa columna, por ende, el recorrido completo es la única opción.

Sin embargo, después de crear el índice el COST no bajó. El optimizador siguió prefiriendo el recorrido completo, y se da por dos razones. La primera es que la consulta pide SUM(total), y total no está en el índice, así que habría que ir a la tabla una vez por cada fila encontrada. La segunda es que esas filas están dispersas por toda la tabla: el clustering factor del índice es 200.001, igual a NUM_ROWS, o sea que cada entrada apunta a un bloque distinto.

Serían miles de lecturas sueltas contra 309 de leer la tabla de corrido. Comprobado forzando el índice con un hint: el COST sube.

Orden de las columnas: La regla general es que van primero las columnas con predicado de igualdad y de última la de rango. Aquí estado = ENTREGADO es igualdad y fecha_pedido es rango, así que (estado, fecha_pedido) permite entrar por un punto exacto y recorrer un tramo continuo, mientras que (fecha_pedido, estado) obliga a recorrer todas las entradas de marzo descartando las que no son ENTREGADO.

Paso 2: sin índice

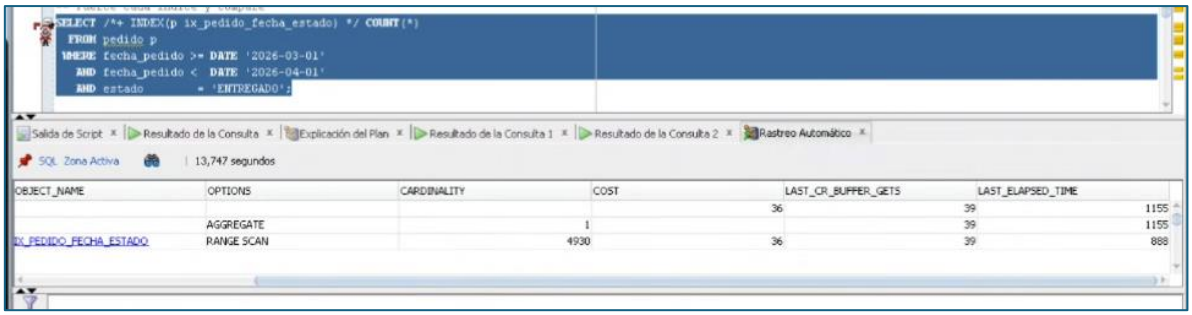
```
PLAN_TABLE_OUTPUT
Plan hash value: 2571571058

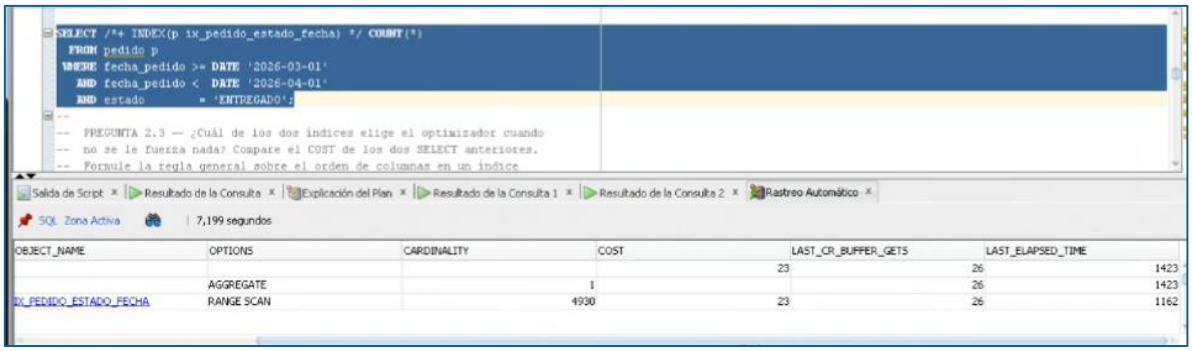
-------------------------------------------------------------
| Id  | Operation          | Name    | Rows  | Cost (%CPU)|
-------------------------------------------------------------
|   0 | SELECT STATEMENT   |         |     1 |   309   (1)|
|   1 |  SORT AGGREGATE    |         |     1 |            |
|   2 |   TABLE ACCESS FULL| PEDIDO  |  5289 |   309   (1)|
-------------------------------------------------------------
```

Paso 4: con índice

	PLAN_TABLE_OUTPUT
1	Plan hash value: 2571571058
2	
3	--
4	| Id | Operation | Name | Rows | Cost (%CPU)|
5	--
6	| 0 | SELECT STATEMENT | | 1 | 309 (1)|
7	| 1 | SORT AGGREGATE | | 1 | |
8	| 2 | TABLE ACCESS FULL| PEDIDO | 4715 | 309 (1)|
9	--

Paso 5: forzando cada índice en distinto orden de columnas





Paso 5: este es el que escoge el optimizador cuando no se le fuerza nada



b. Demuestre por qué un índice sobre una columna deja de usarse cuando se le aplica una función en el predicado, y resuélvalo de dos maneras distintas.

Con ix_producto_nombre creado, la consulta WHERE UPPER(nombre) = TECLADO MECÁNICO MODELO 4001 siguió haciendo recorrido completo. El índice guarda el nombre tal como está en la tabla, y la consulta busca

la versión en mayúsculas, que no está guardada en ninguna parte. Oracle no puede buscar en el índice algo que el índice no contiene, así que lee cada fila, le aplica UPPER y compara.

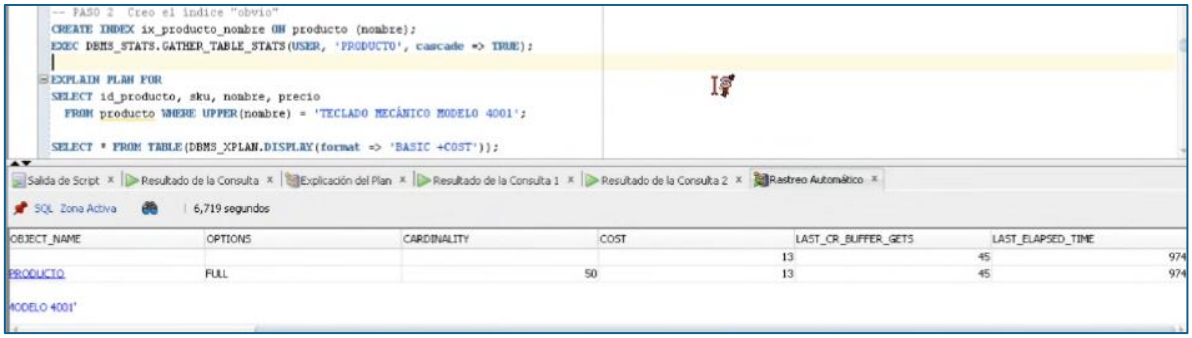
La primera forma de resolverlo es un índice basado en función: CREATE INDEX ix_producto_nombre_upper ON producto (UPPER(nombre)). Ahora el índice sí guarda el valor convertido. La segunda es reescribir el predicado para no tocar la columna, que es lo normal con fechas: TRUNC(fecha_pedido) = DATE 2026-03-15 se cambia por fecha_pedido >= DATE 2026-03-15 AND fecha_pedido < DATE 2026-03-16.

Las dos consultas de fecha devuelven exactamente lo mismo, pero solo la segunda puede usar el índice.

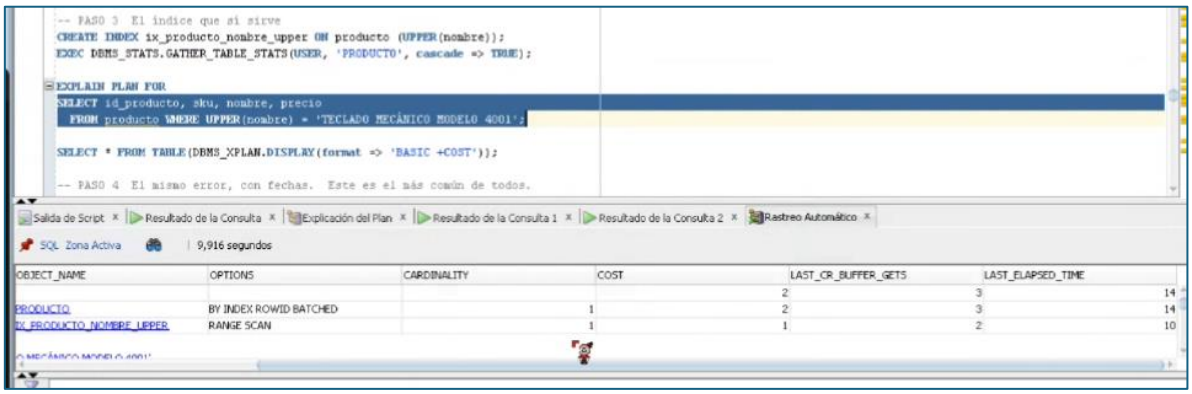
Paso 1: sin índice



Paso 2: con el índice normal sobre la columna



Paso 3: con el índice basado en función



c. Muestre un caso donde el optimizador decide NO usar un índice existente y explique con qué información toma esa decisión.

Con el índice ix_pedido_estado y la misma consulta, cambiando solo el valor: con estado = ENTREGADO no lo usa, y con estado = ANULADO sí.

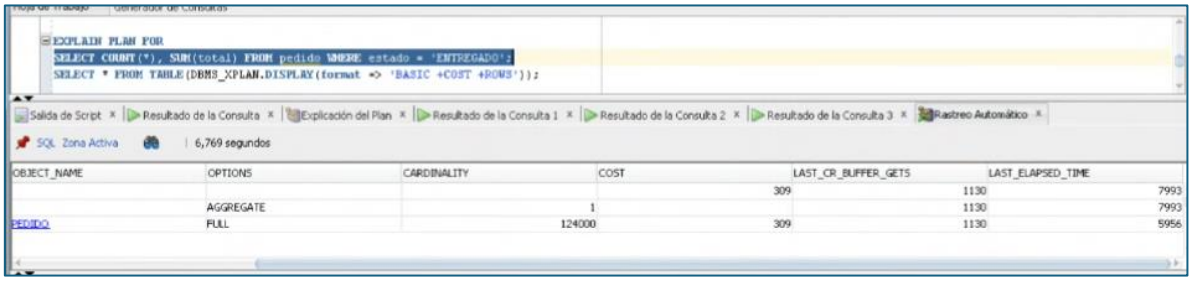
La diferencia es cuántas filas devuelve cada filtro. ENTREGADO es el ~60% de la tabla y ANULADO el ~2%. Cuando el filtro trae casi toda la tabla, ir por el índice y saltar a la tabla una vez por fila sale más caro que leerla de corrido; con el 2% el índice gana.

Con qué información lo decide: Con las estadísticas que recalcula DBMS_STATS: num_distinct y density de la columna, y con el histograma, porque sin él asumiría que cada uno de los 5 estados es el 20% y trataría igual a los dos casos. También pesa el clustering factor del índice.

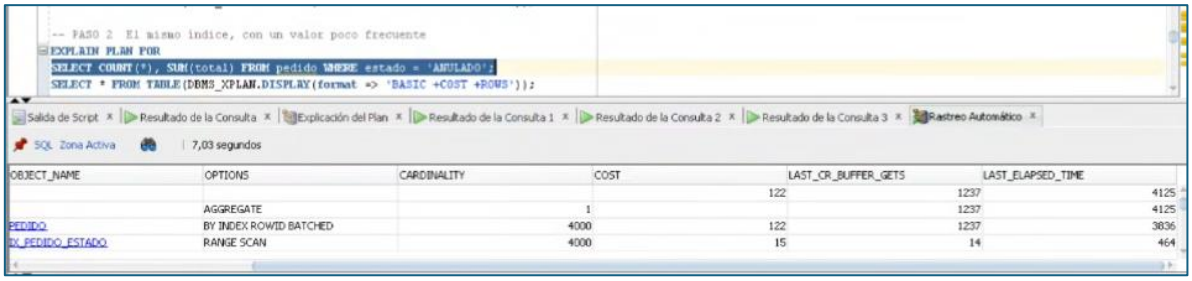
Clustering factor medido:

Índice	Clustering factor
IX_PEDIDO_FECHA_ESTADO	200.001
IX_PEDIDO_ESTADO	5.345
PK_PEDIDO	1.071

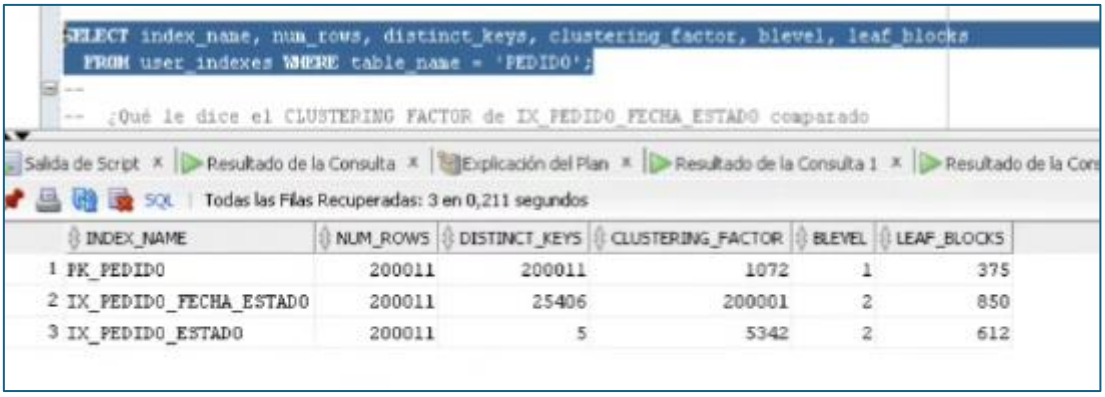
Filtro ENTREGADO



Filtro ANULADO



Clustering factor de los índices



d. Elimine el acceso a la tabla de una consulta agregada agregando la columna faltante al índice, y discuta hasta dónde tiene sentido esa técnica.

Al agregar id_cliente al índice desapareció el TABLE ACCESS BY INDEX ROWID. La técnica se llama índice de cobertura: el índice ya trae fecha_pedido e id_cliente, que es lo que la consulta necesita, sin necesidad de ir a la tabla a buscar nada más.

Cada columna que se agrega engorda el índice, lo hace más lento de recorrer y encarece cada INSERT, UPDATE y DELETE. Vale la pena con una o dos columnas para una consulta frecuente e importante; meter media tabla en el índice ya es hacer una copia que toca mantener sincronizada en cada escritura.

Paso 1: sin el índice de cobertura

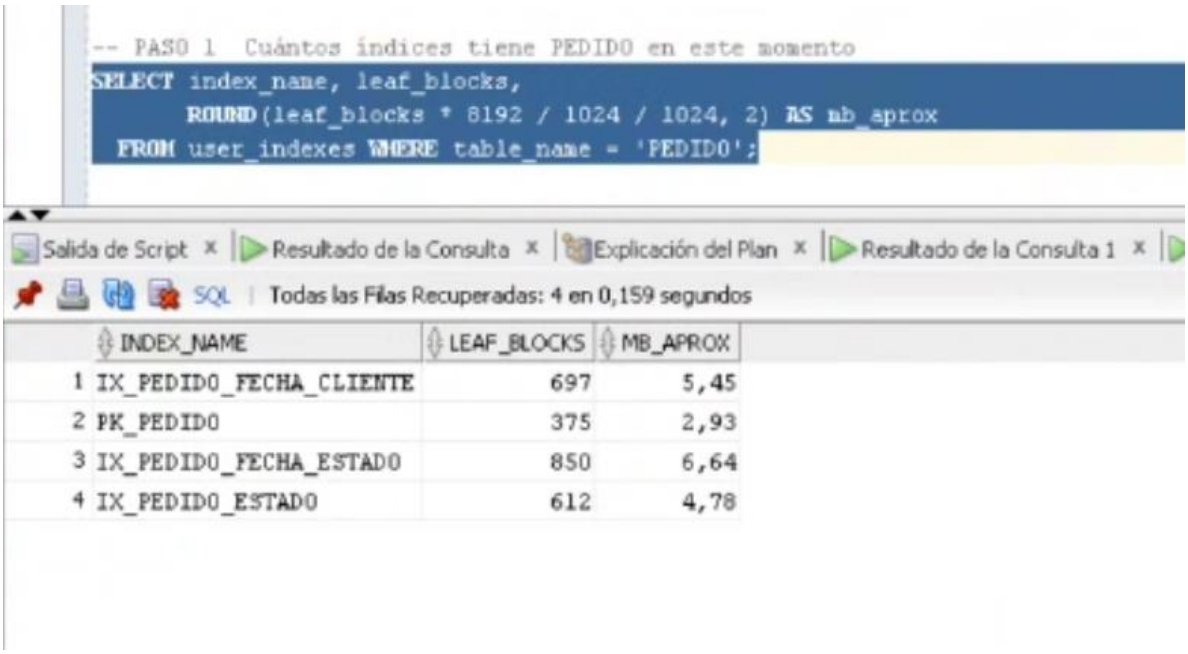


Paso 2: con el índice de cobertura



e. Mida el costo de escritura que impone cada índice y proponga la lista definitiva de índices del esquema, justificando cada uno.

Índices de PEDIDO



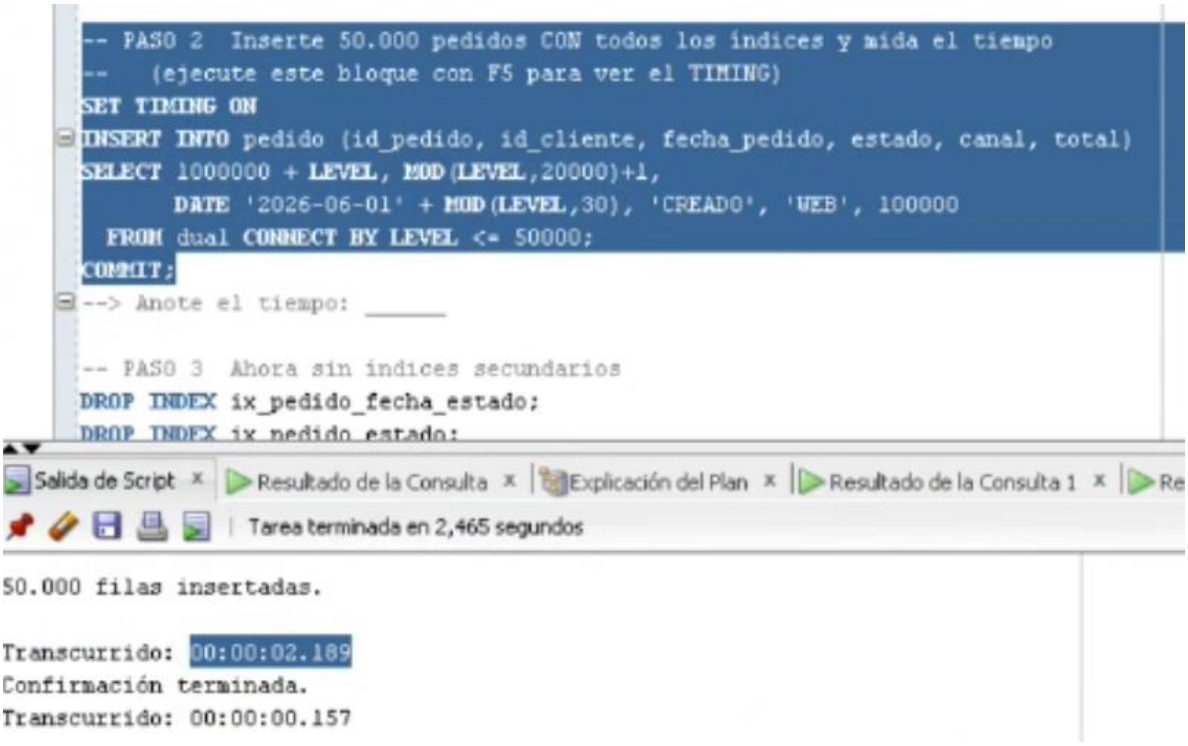
	Tiempo
INSERT de 50.000 filas con los 3 índices	2,189 s

INSERT de 50.000 filas sin índices secundarios	0,373 s

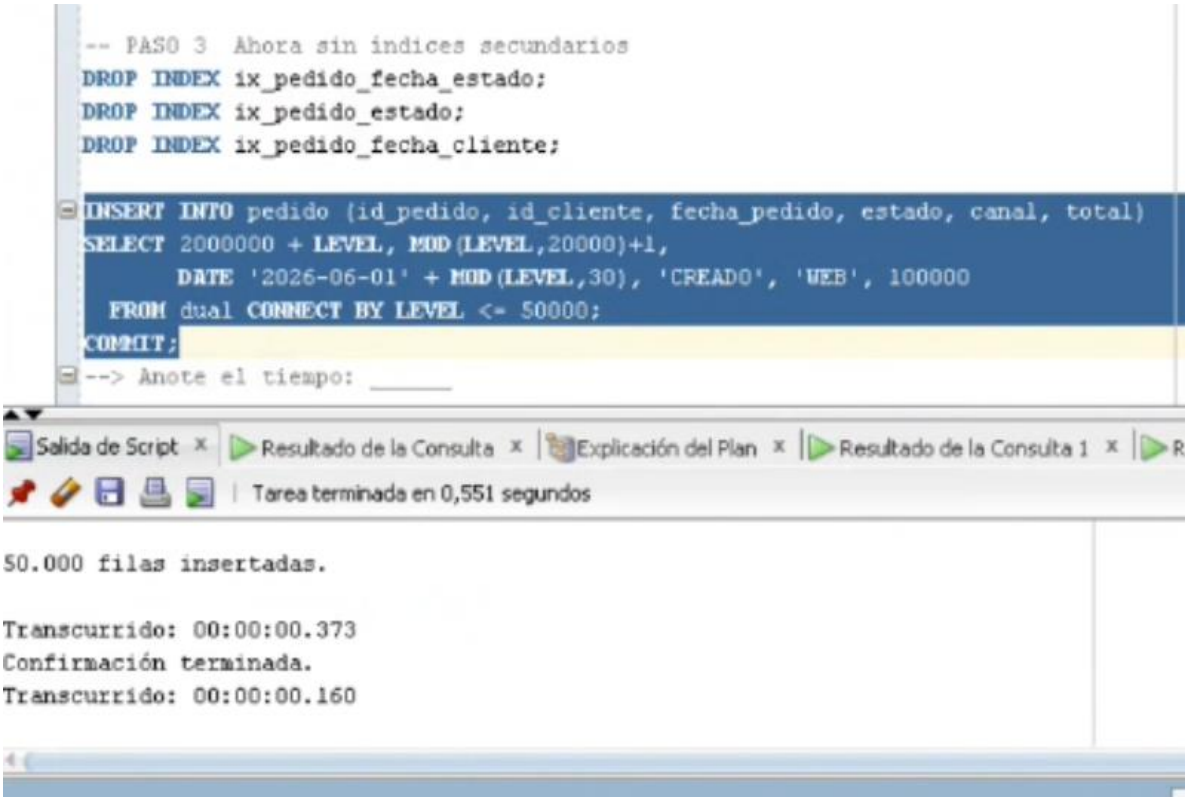
El INSERT tarda casi 6 veces más con los índices: el 80% del tiempo se va en mantenerlos, no en escribir los datos. Son 1,816 s de sobrecosto por 50.000 filas, o sea unos 0,0012 ms por fila por cada índice.

Extrapolando a 8.000 pedidos por hora, cada índice cuesta alrededor de 0,10 s por hora, 2,3 segundos al día, unos 14 minutos de CPU al año, y eso solo en PEDIDO: si el índice fuera sobre PEDIDO_DETALLE sería el doble o el triple, porque cada pedido tiene de 1 a 3 líneas. El costo real no es solo tiempo: también ocupa disco, alarga los backups y consume memoria. Un índice puesto por si acaso que ninguna consulta usa es costo puro sin beneficio, y se paga en cada escritura, para siempre.

Paso 2: inserción con los índices



Paso 3: inserción sin los índices secundarios



Lista definitiva de índices

1. PEDIDO (fecha_pedido, estado). Lo justifican los informes mensuales y el tablero de gerencia, que filtran por rango de fechas. No se puede resolver con otro índice porque ninguno de los existentes contiene la fecha: la llave primaria es por id_pedido. Se prefiere este orden sobre (estado, fecha_pedido) porque también sirve a los informes que filtran únicamente por fecha.

2. PAGO (id_pedido). Lo justifica la llave foránea hacia PEDIDO. La llave primaria de la tabla es por id_pago, así que no cubre esta columna, y sin el índice borrar un pedido bloquea la tabla PAGO completa.

3. PEDIDO_DETALLE (id_producto). Lo justifican las consultas de ventas por producto y la llave foránea hacia PRODUCTO. La llave primaria empieza por id_pedido, de modo que no sirve para buscar por producto.

4. PEDIDO (id_cliente). Lo justifican el historial de pedidos de un cliente, el informe de clientes PREMIUM inactivos y la llave foránea hacia CLIENTE. Tampoco lo cubre la llave primaria, que es por id_pedido.

5. PRODUCTO (UPPER(nombre)). Lo justifica la búsqueda de soporte por nombre. Un índice normal sobre nombre no se usa cuando el predicado le aplica UPPER, porque el índice guarda el valor original y no el convertido.

Índices descartados

PEDIDO (estado). Su valor más consultado, ENTREGADO, devuelve el 62% de la tabla, y en ese caso el optimizador prefiere el recorrido completo y ni siquiera lo utiliza.

PEDIDO (fecha_pedido, id_cliente). Se solapa con el índice 1 y sería un tercer índice sobre la tabla que más escrituras recibe, algo que la medición del costo de escritura desaconseja.

Parte 3. SQL avanzado

a. Ranking de los tres productos más vendidos por categoría, agregando el porcentaje que representa cada uno sobre el total de su categoría.

Se usó DENSE0_RANK() para el puesto de más vendidos y RATIO_TO_REPORT() para el porcentaje. El porcentaje se calcula dentro de la misma subconsulta que el ranking, antes del WHERE puesto <= 3. Si se calculara después de filtrar, los porcentajes sumarían 100% entre los tres productos en lugar de ser sobre la categoría completa.

Las dos funciones van particionadas por id_categoria, así que el ranking se reinicia y el porcentaje se recalcula en cada categoría por separado. La consulta necesita esa subconsulta porque las funciones de ventana no se pueden usar en el WHERE: primero hay que calcularlas y después filtrar sobre el resultado.

Consulta usando RATIO_TO_REPORT

```
  SELECT pr.id_categoria, pr.id_producto, pr.nombre,
         SUM(d.cantidad * d.precio_unit) AS facturado
    FROM pedido p
    JOIN pedido_detalle d  ON d.id_pedido    = p.id_pedido
    JOIN producto       pr ON pr.id_producto = d.id_producto
   WHERE p.fecha_pedido >= DATE '2026-01-01'
     AND p.fecha_pedido <  DATE '2027-01-01'
     AND p.estado <> 'ANULADO'
   GROUP BY pr.id_categoria, pr.id_producto, pr.nombre
)
SELECT id_categoria, nombre, ROUND(facturado) AS facturado, puesto,
       ROUND(pct_categoria, 2) AS pct_categoria
  FROM (SELECT v.*,
               DENSE_RANK() OVER (PARTITION BY id_categoria
                                  ORDER BY facturado DESC) AS puesto,
               100 * RATIO_TO_REPORT(facturado)
                     OVER (PARTITION BY id_categoria)    AS pct_categoria
          FROM ventas v)
 WHERE puesto <= 3
 ORDER BY id_categoria, puesto;
```

Salida de Script | Resultado de la Consulta | Explicación del Plan | Resultado de la Consulta 1

SQL | Todas las Filas Recuperadas: 30 en 0,403 segundos

	ID_CATEGORIA	NOMBRE	FACTURADO	PUESTO	PCT_CATEGORIA
5	101	Teclado mecánico modelo 1851	960601600	2	0,58
6	101	Teclado mecánico modelo 2051	906236400	3	0,55
7	102	Monitor modelo 3422	401283000	1	0,64
8	102	Monitor modelo 1742	390038400	2	0,62
9	102	Monitor modelo 4262	386958600	3	0,62
10	103	Audífonos modelo 3123	934631400	1	0,57
11	103	Audífonos modelo 3323	907970000	2	0,55
12	103	Audífonos modelo 563	885022600	3	0,54
13	104	Parlante modelo 1234	375757200	1	0,63
14	104	Parlante modelo 2914	373573200	2	0,62

b. Evolución mensual de ventas con acumulado del año, variación contra el mes anterior y promedio móvil de tres meses.

Se agregó al modelo la columna AVG(ventas) OVER (ORDER BY mes ROWS BETWEEN 2 PRECEDING AND CURRENT ROW). En enero el promedio es de un solo mes y en febrero de dos, porque la ventana todavía no tiene tres filas atrás.

La diferencia entre ROWS y RANGE se muestra en que ROWS cuenta filas físicas: las 2 anteriores y la actual son tres filas. RANGE trabaja sobre los valores de la columna del ORDER BY, así que incluye todas las filas empatadas con la actual. Solo dan distinto cuando hay empates: si hubiera dos filas del mismo mes, con ROWS cada una vería su propio acumulado y con RANGE las dos verían el mismo total. En el modelo no se nota porque el GROUP BY deja un solo mes por fila.

Evolución mensual con acumulado, variación y promedio móvil

```
WITH mensual AS (
  SELECT TRUNC(fecha_pedido, 'MM') AS mes,
         SUM(total)                AS ventas,
         COUNT(*)                  AS pedidos
    FROM pedido
   WHERE fecha_pedido >= DATE '2026-01-01'
     AND fecha_pedido <  DATE '2027-01-01'
     AND estado <> 'ANULADO'
   GROUP BY TRUNC(fecha_pedido, 'MM')
)
SELECT TO_CHAR(mes,'YYYY-MM')                          AS mes,
       pedidos,
       ROUND(ventas)                                   AS ventas,
       ROUND(SUM(ventas) OVER (ORDER BY mes
             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)) AS acumulado,
       ROUND(LAG(ventas) OVER (ORDER BY mes))          AS mes_anterior,
       ROUND(100 * (ventas - LAG(ventas) OVER (ORDER BY mes))
                 / NULLIF(LAG(ventas) OVER (ORDER BY mes),0), 1) AS var_pct,
              ROUND(AVG(ventas) OVER (ORDER BY mes
             ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)) AS prom_movil_3m
  FROM mensual
```

Explicación del Plan

Salida de Script | Resultado de la Consulta | Explicación del Plan | Resultado de la Consulta 1 | Resultado de la C

Todas las Filas Recuperadas: 12 en 0,12 segundos

	MES	PEDIDOS	VENTAS	ACUMULADO	MES_ANTERIOR	VAR_PCT	PROM_MOVIL_3M
1	2026-01	8330	93998039800	93998039800	(null)	(null)	93998039800
2	2026-02	7508	85338658700	179336698500	93998039800	-9,2	89668349250
3	2026-03	8330	94322759500	273659458000	85338658700	10,5	91219819333
4	2026-04	8056	91441588200	365101046200	94322759500	-3,1	90367668800
5	2026-05	8330	93827057200	458928103400	91441588200	2,6	93197134967
6	2026-06	8056	91606458000	550534561400	93827057200	-2,4	92291701133
7	2026-07	8302	93660156300	644194717700	91606458000	2,2	93031223833
8	2026-08	8326	94447593200	738642310900	93660156300	0,8	93238069167
9	2026-09	8061	90767536600	829409847500	94447593200	-3,9	92958428700

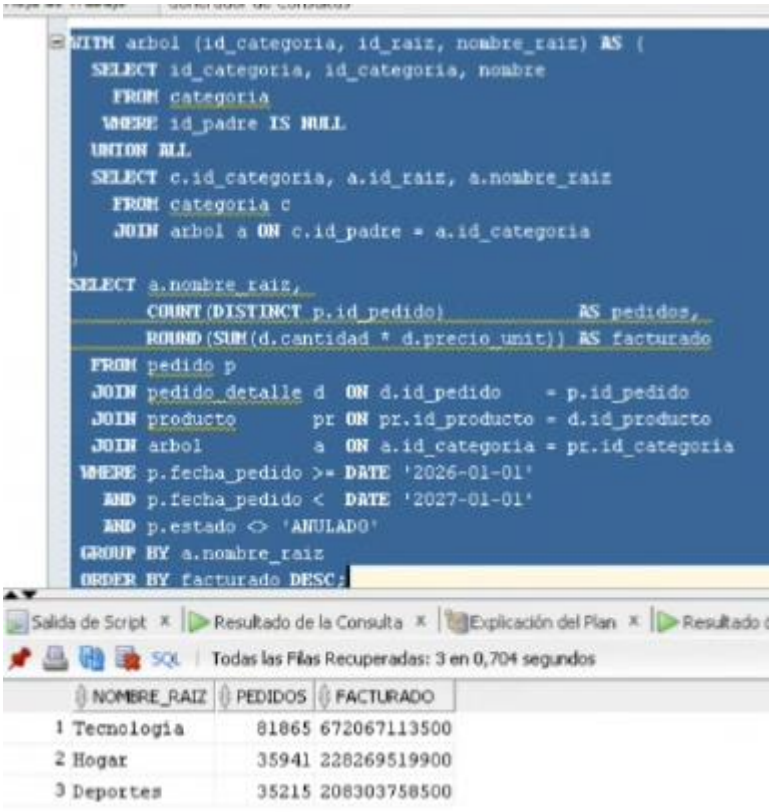
c. Facturación por categoría raíz, resolviendo la jerarquía de tres niveles con una expresión de tabla común recursiva.

Se resolvió con una CTE recursiva en la que cada categoría hija hereda el id_raiz y el nombre_raiz de su padre en lugar de recalcularlos, de modo que al llegar al nivel 3 la categoría ya trae pegada su raíz. Devuelve 3 filas: Tecnología, Hogar y Deportes.

En este ejercicio, se ve que CTE recursiva y CONNECT BY funcionan distinto. La CTE trabaja por niveles: arranca con una consulta base, las categorías sin padre, y esa consulta se vuelve a ejecutar contra el resultado anterior, sacando un nivel completo en cada vuelta. CONNECT BY recorre rama por rama: baja por un hijo hasta el fondo, vuelve y sigue con el siguiente.

CONNECT BY siempre sabe a qué profundidad va y por dónde pasó al ir bajando por la rama, y por eso entrega LEVEL sin que uno haga nada.

Facturación de 2026 por categoría raíz



```
WITH arbol (id_categoria, id_raiz, nombre_raiz) AS (
  SELECT id_categoria, id_categoria, nombre
    FROM categoria
   WHERE id_padre IS NULL
  UNION ALL
  SELECT c.id_categoria, a.id_raiz, a.nombre_raiz
    FROM categoria c
    JOIN arbol a ON c.id_padre = a.id_categoria
)
SELECT a.nombre_raiz,
       COUNT(DISTINCT p.id_pedido)            AS pedidos,
       ROUND(SUM(d.cantidad * d.precio_unit)) AS facturado
  FROM pedido p
  JOIN pedido_detalle d  ON d.id_pedido    = p.id_pedido
  JOIN producto       pr ON pr.id_producto = d.id_producto
  JOIN arbol          a  ON a.id_categoria = pr.id_categoria
 WHERE p.fecha_pedido >= DATE '2026-01-01'
   AND p.fecha_pedido <  DATE '2027-01-01'
   AND p.estado <> 'ANULADO'
 GROUP BY a.nombre_raiz
 ORDER BY facturado DESC;
```

Salida de Script | Resultado de la Consulta | Explicación del Plan | Resultado

Todas las Filas Recuperadas: 3 en 0,704 segundos

	NOMBRE_RAIZ	PEDIDOS	FACTURADO
1	Tecnologia	81865	672067113500
2	Hogar	35941	228269519900
3	Deportes	35215	208303758500

d. Aplicación de los ajustes de bodega al inventario mediante MERGE, incluyendo el borrado de las filas que queden en cero.

Se agregó la cláusula DELETE WHERE i.disponible = 0 AND i.id_bodega <> 1 dentro del WHEN MATCHED, después del UPDATE.

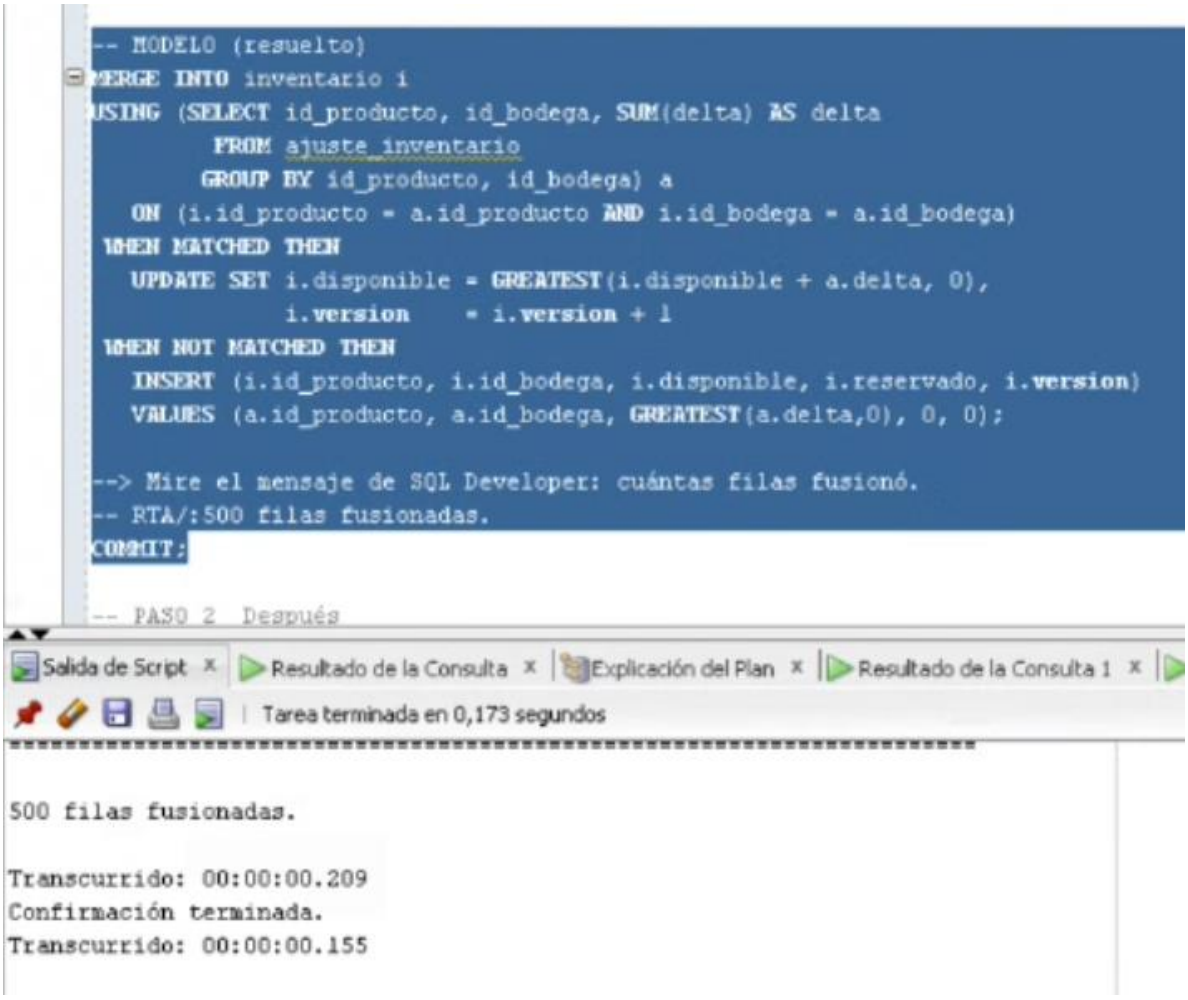
Por qué se agrupa el origen dentro del USING: Porque la tabla de ajustes puede traer varios movimientos del mismo producto en la misma bodega, por ejemplo una merma y un ingreso. MERGE necesita saber con qué fila actualizar cada fila del inventario, y si le llegan dos no sabe cuál escoger: falla toda la sentencia con ORA-30926. Al agrupar con SUM(delta) queda una sola fila por clave, que además es lo correcto desde el negocio.

Sobre el GREATEST(..., 0). Está escondiendo un problema. Si el inventario tiene 3 unidades y llega una merma de 10, guarda 0 y nadie se entera de que faltaron 7; la diferencia entre lo que dice el sistema y lo que hay en bodega se pierde sin dejar rastro. En un sistema real dejaríamos que el ajuste falle, apoyándonos en la restricción CHECK (disponible >= 0) que la tabla ya tiene, separando los ajustes que no cuadran en una tabla de excepciones con su motivo y avisando a bodega para conteo físico. Un dato que no cuadra se reporta, no se redondea.

Antes del MERGE



Ejecución del MERGE



Después del MERGE



e. Informe de clientes PREMIUM sin compras en los últimos ciento veinte días, con días de inactividad y facturación histórica.

Comercial quiere una lista de clientes que se están perdiendo: los PREMIUM que llevan más de 120 días sin comprar. Y de cada uno quiere saber cuántos días lleva inactivo y cuánto facturó históricamente, para saber a quién vale la pena llamar. Se resolvió con una CTE que calcula por cliente su última compra, cuántos pedidos tiene y cuánto ha facturado, y después se filtran los PREMIUM con más de 120 días sin comprar.

Se agregó la condición fecha_pedido <= SYSDATE porque los datos del setup llegan hasta finales de 2026 y hoy es septiembre de 2026, así que hay pedidos con fecha futura. Sin ese filtro los días de inactividad salían negativos para todos los clientes.

El resultado devuelve cero, pues el máximo de inactividad real es de 89 días, por debajo del umbral de 120. Cada cliente tiene unos diez pedidos repartidos en dos años, o sea que compra cada 70 días y nunca deja un hueco de cuatro meses. Se verificó bajando el umbral a 60 días, donde sí devuelve resultados.

Consulta con el umbral de 120 días

```
WITH actividad AS (
  SELECT id_cliente,
         MAX(fecha_pedido) AS ultima_compra,
         COUNT(*)          AS pedidos_historicos,
         SUM(total)        AS facturacion_historica
    FROM pedido
   WHERE estado <> 'ANULADO'
     AND fecha_pedido <= SYSDATE
   GROUP BY id_cliente
)
SELECT c.id_cliente,
       c.nombre,
       c.ciudad,
       TRUNC(a.ultima_compra)           AS ultima_compra,
       TRUNC(SYSDATE - a.ultima_compra) AS dias_inactivo,
       a.pedidos_historicos,
       ROUND(a.facturacion_historica)   AS facturado_historico
  FROM cliente   c
  JOIN actividad a ON a.id_cliente = c.id_cliente
 WHERE c.segmento = 'PREMIUM'
   AND a.ultima_compra < SYSDATE - 120
 ORDER BY dias_inactivo DESC;
```

Salida de Script | Resultado de la Consulta | Todas las Filas Recuperadas: 0 en 0,109 segundos

ID_CLIENTE	NOMBRE	CIUDAD	ULTIMA_...	DIAS_INA...	PEDIDOS...	FACTURA...

Verificación bajando el umbral

```
WITH actividad AS (
  SELECT id_cliente,
         MAX(fecha_pedido) AS ultima_compra,
         COUNT(*)          AS pedidos_historicos,
         SUM(total)        AS facturacion_historica
    FROM pedido
   WHERE estado <> 'ANULADO'
     AND fecha_pedido <= SYSDATE
   GROUP BY id_cliente
)
SELECT c.id_cliente,
       c.nombre,
       c.ciudad,
       TRUNC(a.ultima_compra)           AS ultima_compra,
       TRUNC(SYSDATE - a.ultima_compra) AS dias_inactivo,
       a.pedidos_historicos,
       ROUND(a.facturacion_historica)   AS facturado_historico
  FROM cliente   c
  JOIN actividad a ON a.id_cliente = c.id_cliente
 WHERE c.segmento = 'PREMIUM'
   AND a.ultima_compra < SYSDATE - 60
 ORDER BY dias_inactivo DESC;
```

Salida de Script | Resultado de la Consulta | Se han recuperado 350 filas en 0,617 segundos

	ID_CLIENTE	NOMBRE	CIUDAD	ULTIMA_COMPRA	DIAS_INACTIVO	PEDIDOS_HISTORICOS	FACTURADO_HISTORICO
1	1791	Cliente 1791	Barranquilla	15/06/26	89	8	31165600
2	4131	Cliente 4131	Barranquilla	15/06/26	89	8	54289400
3	9091	Cliente 9091	Medellín	15/06/26	89	8	22818700
4	10701	Cliente 10701	Barranquilla	15/06/26	89	8	119095600
5	12891	Cliente 12891	Barranquilla	15/06/26	89	8	116913200
6	14641	Cliente 14641	Medellín	15/06/26	89	8	40016000

Parte 4. Reto integrador

a. Identifique cuatro defectos en la secuencia que ejecuta la aplicación.

Defecto 1 - concurrencia: El SELECT disponible lee el stock pero no lo bloquea, y después el UPDATE inventario SET disponible = 7 escribe un número fijo que la aplicación calculó aparte. Si otra confirmación del mismo producto se cuela en el medio, la segunda le escribe encima a la primera y esa venta desaparece. Produce el síntoma 1 del reporte: se venden más unidades de las que hay. Con 200 confirmaciones por minuto esos cruces pasan todo el tiempo y el inventario del sistema termina siempre por encima del físico.

Defecto 2 – transaccionalidad: Hay tres COMMIT, uno por cada línea. Cada uno es un punto sin retorno, así que la confirmación deja de ser una sola operación. Si el proceso se cae después del primero, el producto 100 queda descontado, el 200 no, y el pedido sigue en CREADO, sin posibilidad de deshacer. Produce el síntoma 2: pedidos a medias que nadie sabe cuáles son. En producción basta una caída o un timeout para dejar uno inconsistente.

Defecto 3 - filas inadvertidas: Ninguna sentencia revisa cuántas filas afectó ni si alcanzaba el stock, y el UPDATE pedido tampoco verifica que estuviera en CREADO, así que un pedido se puede pagar dos veces. Como el valor lo calcula la aplicación y lo escribe ya listo, la restricción CHECK (disponible >= 0), que sí existe en la tabla, nunca llega a evaluarse: le llega un número positivo y lo aprueba. La protección está puesta pero la forma de escribir el UPDATE la esquiva. Este defecto no causa un síntoma propio, lo que hace es tapar los otros dos; por eso el problema apareció contando en bodega y no en un log de errores.

Defecto 4 – rendimiento: La aplicación manda un SELECT y un UPDATE por cada línea del pedido, en viajes separados, y confirma en cada paso. Va de a una fila cuando podría hacerlo todo en una sola sentencia. Con 200 confirmaciones por minuto son cientos de sentencias y de COMMIT por minuto, y cada COMMIT obliga a escribir en disco. Además, la secuencia dura más, que es justo cuando el defecto 1 tiene chance de aparecer.

b. Reescriba la secuencia usando solamente SQL y demuestre la corrección.

Conexión A: Tenemos un producto 100 y el 200 en 10 unidades y el pedido en CREADO, A ejecuta el SELECT ... FOR UPDATE y recibe las dos filas de inmediato; desde ahí son suyas hasta que confirme. El MERGE responde 2 filas fusionadas y el UPDATE pedido responde 1 fila actualizada, porque el pedido sí estaba en CREADO. Con el COMMIT guarda todo junto y el inventario queda en 7 y 8.

Conexión B: B arranca mientras A todavía no confirma, así que su SELECT ... FOR UPDATE se queda esperando. Cuando A hace COMMIT, B recibe las filas ya con los valores nuevos, 7 y 8, y su MERGE descuenta sobre esos y no sobre los 10 que habría leído si nadie la hubiera detenido. Al llegar al UPDATE pedido obtiene 0 filas actualizadas, porque el pedido ya está en PAGADO y la condición AND estado = CREADO no se cumple. B hace ROLLBACK y su descuento se deshace.

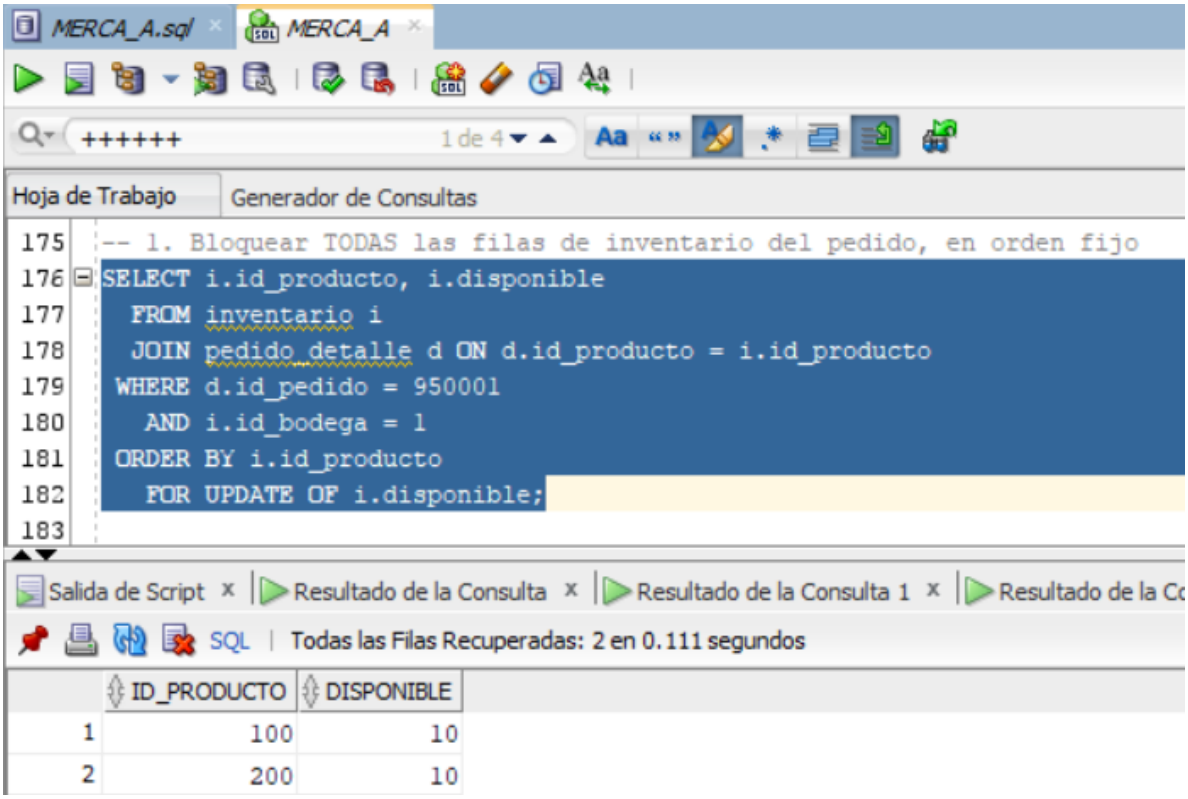
Esto demuestra que, con la secuencia original las dos sesiones habrían escrito el mismo número calculado por separado, perdiendo una venta y pagando el pedido dos veces. Aquí el bloqueo obligó a la segunda a esperar, el descuento relativo la hizo trabajar sobre el dato vigente, y la condición del estado le avisó que ya no había nada que confirmar. Nada se perdió, nada se aplicó dos veces, y el problema quedó registrado en un contador de filas en lugar de pasar inadvertido.

Secuencia corregida

```sql
-- 1. Bloquear TODAS las filas de inventario del pedido, en orden fijo
SELECT i.id_producto, i.disponible
  FROM inventario i
  JOIN pedido_detalle d ON d.id_producto = i.id_producto
 WHERE d.id_pedido = 950001
   AND i.id_bodega = 1
 ORDER BY i.id_producto
   FOR UPDATE OF i.disponible;
-- 2. Descontar TODO el pedido con UNA sola sentencia de conjunto
MERGE INTO inventario i
USING (SELECT id_producto, SUM(cantidad) AS cantidad
         FROM pedido_detalle
        WHERE id_pedido = 950001
        GROUP BY id_producto) d
   ON (i.id_producto = d.id_producto AND i.id_bodega = 1)
 WHEN MATCHED THEN
   UPDATE SET i.disponible = i.disponible - d.cantidad,
              i.version    = i.version + 1;
-- Debe decir "2 filas fusionadas". Si dice menos, ROLLBACK.
-- 3. Marcar el pedido SOLO si estaba en CREADO
UPDATE pedido
   SET estado = 'PAGADO'
 WHERE id_pedido = 950001
   AND estado    = 'CREADO';
-- Debe decir "1 fila actualizada". Si dice 0, alguien ya lo pagó: ROLLBACK.
-- 4. Un solo COMMIT
COMMIT;
```

Cómo se probó

Conexión A: bloqueo inicial



Conexión B: intenta el mismo bloqueo y queda esperando

```
-- 1. Bloquear TODAS las filas de inventario del pedido, en orden fijo
SELECT i.id_producto, i.disponible
  FROM inventario i
  JOIN pedido_detalle d ON d.id_producto = i.id_producto
 WHERE d.id_pedido = 950001
   AND i.id_bodega = 1
 ORDER BY i.id_producto
   FOR UPDATE OF i.disponible;

-- 2. Descontar TODO el pedido con UNA sola sentencia de conjunto
MERGE INTO inventario i
USING (SELECT id_producto, SUM(cantidad) AS cantidad
```

alida de Script x | Resultado de la Consulta x

SQL | Resultado de la Consulta

Conexión A: MERGE

```
184  -- 2. Descontar TODO el pedido con UNA sola sentencia de conjunto
185  MERGE INTO inventario i
186  USING (SELECT id_producto, SUM(cantidad) AS cantidad
187           FROM pedido_detalle
188          WHERE id_pedido = 950001
189          GROUP BY id_producto) d
190     ON (i.id_producto = d.id_producto AND i.id_bodega = 1)
191   WHEN MATCHED THEN
192     UPDATE SET i.disponible = i.disponible - d.cantidad,
193                i.version    = i.version + 1;
194  --> Debe decir "2 filas fusionadas". Si dice menos, ROLLBACK.
195
196  -- 3. Marcar el pedido SOLO si estaba en CREADO
```

Salida de Script x | Resultado de la Consulta x | Resultado de la Consulta 1 x | Resultado de la Cons

Tarea terminada en 0.209 segundos

```
-  cada categoría hoja, y únala con las ventas.
Informe de error -
Comando desconocido

>>Query Run In:Resultado de la Consulta 10
Transcurrido: 00:00:00.103

2 filas fusionadas.
```

Conexión A: UPDATE del pedido y COMMIT

```
196  -- 3. Marcar el pedido SOLO si estaba en CREADO
197  UPDATE pedido
198     SET estado = 'PAGADO'
199   WHERE id_pedido = 950001
200     AND estado    = 'CREADO';
201  --> Debe decir "1 fila actualizada". Si dice 0, alguien ya lo pagó: ROLLBACK.
202
203  -- 4. Un solo COMMIT
204  COMMIT;
205  --
```

Salida de Script | Resultado de la Consulta | Resultado de la Consulta 1 | Resultado de la Consulta 2

Tarea terminada en 0.339 segundos

```
Transcurrido: 00:00:00.195

1 fila actualizadas.

Transcurrido: 00:00:00.133
Confirmación terminada.
Transcurrido: 00:00:00.162
```

Conexión B: deja de esperar y ve los valores ya descontados

```
SELECT i.id_producto, i.disponible
  FROM inventario i
  JOIN pedido_detalle d ON d.id_producto = i.id_producto
 WHERE d.id_pedido = 950001
   AND i.id_bodega = 1
 ORDER BY i.id_producto
   FOR UPDATE OF i.disponible;

-- 2. Descontar TODO el pedido con UNA sola sentencia
MERGE INTO inventario i
USING (SELECT id_producto, SUM(cantidad) AS cantidad
         FROM pedido_detalle
        WHERE id_pedido = 950001
        GROUP BY id_producto) d
   ON (i.id_producto = d.id_producto AND i.id_bodega =
 WHEN MATCHED THEN
   UPDATE SET i.disponible = i.disponible - d.cantidad
```

da de Script | Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 2 en 110,35 segundos

	ID_PRODUCTO	DISPONIBLE
1	100	7
2	200	8

Conexión B: MERGE y UPDATE. En vez de seguir descontando se hace ROLLBACK, para que quede en 7 y 8

```
-- 2. Descontar TODO el pedido con UNA sola sentencia de conjunto
MERGE INTO inventario i
USING (SELECT id_producto, SUM(cantidad) AS cantidad
         FROM pedido_detalle
        WHERE id_pedido = 950001
        GROUP BY id_producto) d
   ON (i.id_producto = d.id_producto AND i.id_bodega = 1)
 WHEN MATCHED THEN
   UPDATE SET i.disponible = i.disponible - d.cantidad,
              i.version    = i.version + 1;
--> Debe decir "2 filas fusionadas". Si dice menos, ROLLBACK.

-- 3. Marcar el pedido SOLO si estaba en CREADO
UPDATE pedido
   SET estado = 'PAGADO'
 WHERE id_pedido = 950001
   AND estado    = 'CREADO';
--> Debe decir "1 fila actualizada". Si dice 0, alguien ya lo pagó: ROLLBACK
ROLLBACK;
-- 4. Un solo COMMIT
```

Conexión A: verifica los productos y quedan en 7 y 8

```
175  -- 1. Bloquear TODAS las filas de inventario del pedido, en orden fijo
176  SELECT i.id_producto, i.disponible
177    FROM inventario i
178    JOIN pedido_detalle d ON d.id_producto = i.id_producto
179   WHERE d.id_pedido = 950001
180     AND i.id_bodega = 1
181   ORDER BY i.id_producto
182     FOR UPDATE OF i.disponible;
183
```

Salida de Script x | Resultado de la Consulta x | Resultado de la Consulta 1 x | Resultado de la

SQL | Todas las Filas Recuperadas: 2 en 0.145 segundos

	ID_PRODUCTO	DISPONIBLE
1	100	7
2	200	8

Si el siguiente descuento dejara el inventario en negativo, se produce un error

```
SELECT i.id_producto, i.disponible
  FROM inventario i
  JOIN pedido_detalle d ON d.id_producto = i.id_producto
 WHERE d.id_pedido = 950001
   AND i.id_bodega = 1
 ORDER BY i.id_producto
   FOR UPDATE OF i.disponible;
```

Salida de Script | Resultado de la Consulta | Resultado de la Consulta 1

SQL | Todas las Filas Recuperadas: 2 en 0,078 segundos

	ID_PRODUCTO	DISPONIBLE
1	100	1
2	200	4

El CHECK de la tabla se activa y devuelve ORA-02290

```
Error que empieza en la línea: 185 del comando :
MERGE INTO inventario i
USING (SELECT id_producto, SUM(cantidad) AS cantidad
         FROM pedido_detalle
        WHERE id_pedido = 950001
        GROUP BY id_producto) d
   ON (i.id_producto = d.id_producto AND i.id_bodega = 1)
```

c. Optimice la consulta del tablero.

	COST	Consistent gets	Acceso a PEDIDO
Plan actual (con TO_CHAR)	778	2.737	TABLE ACCESS FULL
Plan reescrito (con rango)	775	2.747	TABLE ACCESS FULL
Plan forzando el índice (hint)	773	2.747	INDEX RANGE SCAN

Qué impedía el uso del índice. En la sección de predicados del plan original aparece TO_CHAR(INTERNAL_FUNCTION(P.FECHA_PEDIDO),YYYY-MM)=2026-03. El TO_CHAR le cambia el valor a la columna: el índice guarda la fecha tal cual, no convertida a texto, así que Oracle no puede buscar ahí el valor y le toca leer cada fila, convertirla y comparar.

Al reescribir el filtro como rango, los predicados quedan sobre la columna limpia y el índice ya es utilizable. La diferencia de fondo es que antes el índice era imposible de usar y ahora es posible pero no conveniente. El TO_CHAR bloqueaba cualquier optimización, incluidas las futuras.

Por qué no se propone otro índice: Uno de cobertura sobre (fecha_pedido, estado, id_cliente, total) sí bajaría el costo porque evitaría tocar la tabla. No se propone porque PEDIDO es la tabla que más escrituras recibe y en la Parte 2 medimos que los índices multiplican por 6 el tiempo de inserción. Para un tablero que se abre unas pocas veces al día, no compensa.

Plan original, con TO_CHAR

```
SELECT c.ciudad,
       COUNT(DISTINCT p.id_pedido)              AS pedidos,
       ROUND(SUM(d.cantidad * d.precio_unit))   AS facturado,
       ROUND(AVG(p.total))                      AS ticket
  FROM pedido         p
  JOIN cliente        c ON c.id_cliente  = p.id_cliente
  JOIN pedido_detalle d ON d.id_pedido   = p.id_pedido
 WHERE TO_CHAR(p.fecha_pedido, 'YYYY-MM') = '2026-03'
   AND p.estado <> 'ANULADO'
 GROUP BY c.ciudad
 ORDER BY facturado DESC;
```

Salida de Script | Resultado de la Consulta | Resultado de la Consulta 1 | Resultado de la Consulta 2 | Resultado de la Consulta 3 | Resultado de la Consulta 4 | Resultado de la Consulta 5 | Rastr...

SQL Zona Activa | 10,712 segundos

OBJECT_NAME	OPTIONS	CARDINALITY	COST	LAST_CR_BUFFER_GETS	LAST_ELAPSED_TIME
Access Predicates					
C.ID_CLIENTE=P.ID_CLIENTE					
ABLE ACCE PEDIDO	FULL	8330	312	1130	
Filter Predicates					
AND					
TO_CHAR(INTERNAL_FUNCTION(P.FECHA_PEDIDO),'YYYY-MM')='2026-03'					
P.ESTADO<>'ANULADO'					
ABLE ACCE CLIENTE	FULL	20000	49	166	
PK_PEDIDO_DET	RANGE SCAN			0	
ss Predicates					
.ID_PEDIDO=P.ID_PEDIDO					
PEDIDO_DETALLE	BY INDEX ROWID	2	412	0	
PEDIDO_DETALLE	FULL	400003	412	1451	

Plan de la consulta reescrita, con los predicados de rango

```
-- Consulta corregida
SELECT c.ciudad,
       COUNT(DISTINCT p.id_pedido)              AS pedidos,
       ROUND(SUM(d.cantidad * d.precio_unit))   AS facturado,
       ROUND(AVG(p.total))                      AS ticket
  FROM pedido         p
  JOIN cliente        c ON c.id_cliente  = p.id_cliente
  JOIN pedido_detalle d ON d.id_pedido   = p.id_pedido
 WHERE p.fecha_pedido >= DATE '2026-03-01'
   AND p.fecha_pedido <  DATE '2026-04-01'
   AND p.estado <> 'ANULADO'
 GROUP BY c.ciudad
 ORDER BY facturado DESC;
```

Salida de Script | Resultado de la Consulta | Resultado de la Consulta 1 | Resultado de la Consulta 2 | Resultado de la Consulta 3 | Resultado de la Consulta 4 | Resultado de la Consulta 5 | Rastr...

SQL Zona Activa | 10,257 segundos

OBJECT_NAME	OPTIONS	CARDINALITY	COST	LAST_CR_BUFFER_GETS	LAST_ELAPSED_TIME
ss Predicates					
ID_CLIENTE=P.ID_CLIENTE					
ACCE PEDIDO	FULL	7809	309	1130	
Filter Predicates					
AND					
P.FECHA_PEDIDO>=TO_DATE(' 2026-03-01 00:00:00', 'syyyy-mm-dd hh24:mi:ss')					
P.FECHA_PEDIDO<TO_DATE(' 2026-04-01 00:00:00', 'syyyy-mm-dd hh24:mi:ss')					
P.ESTADO<>'ANULADO'					
ACCE CLIENTE	FULL	20000	49	166	
PK_PEDIDO_DET	RANGE SCAN			0	
redicates					
PEDIDO=P.ID_PEDIDO					
PEDIDO_DETALLE	BY INDEX ROWID	2	412	0	

Plan forzando el índice con el hint

```
EXPLAIN PLAN FOR
SELECT /*+ INDEX(p ix_pedido_fecha_estado) */
       c.ciudad, COUNT(DISTINCT p.id_pedido),
       ROUND(SUM(d.cantidad * d.precio_unit)), ROUND(AVG(p.total))
  FROM pedido p
  JOIN cliente c ON c.id_cliente = p.id_cliente
  JOIN pedido_detalle d ON d.id_pedido = p.id_pedido
 WHERE p.fecha_pedido >= DATE '2026-03-01'
   AND p.fecha_pedido <  DATE '2026-04-01'
   AND p.estado <> 'ANULADO'
 GROUP BY c.ciudad;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(format => 'BASIC +COST +ROWS'));
```

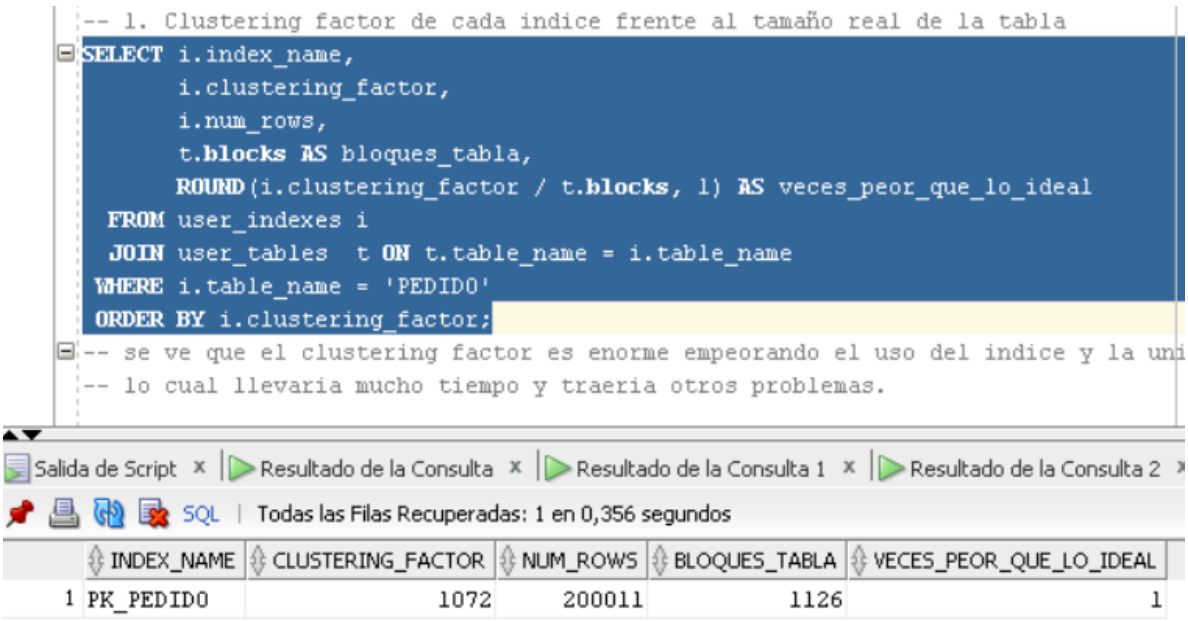
Hoja de Trabajo | Generador de Consultas

Salida de Script | Resultado de la Consulta | Resultado de la Consulta 1 | Resultado de la Consulta 2 | Resultado de la Consulta 3 | Resultado de la Consulta 4 | Resultado de la Consulta 5 | Rastr...

SQL Zona Activa | 10,456 segundos

OBJECT_NAME	OPTIONS	CARDINALITY	COST	LAST_CR_BUFFER_GETS	LAST_ELAPSE
AND					
P.FECHA_PEDIDO>=TO_DATE(' 2026-03-01 00:00:00', 'syyyy-mm-dd hh24:mi:ss')					
P.FECHA_PEDIDO<TO_DATE(' 2026-04-01 00:00:00', 'syyyy-mm-dd hh24:mi:ss')					
P.ESTADO<>'ANULADO'					
TABLE ACCESS CLIENTE	FULL	20000	49	166	
INDEX PK_PEDIDO_DET	RANGE SCAN			0	
Access Predicates					
D.ID_PEDIDO=P.ID_PEDIDO					
ABLE ACCESS PEDIDO_DETALLE	BY INDEX ROWID	2	412	0	
E ACCESS PEDIDO_DETALLE	FULL	400003	412	1451	
er_tab"					

Clustering factor de los índices de PEDIDO



d. Redacte un párrafo dirigido al jefe de Operaciones explicando qué práctica debe cambiar la organización para que este problema no se repita.

El problema no es un pedido en particular, sino dónde están guardadas las reglas del negocio. Hay tres que no se pueden romper nunca: que el inventario no quede en negativo, que un pedido no se pague dos veces, y que descontar el inventario y marcar el pedido pasen siempre juntos o ninguno de los dos. Hoy esas reglas están escritas dentro del código de cada canal, la tienda web, la aplicación, el call center y los procesos de bodega, y todos escriben sobre las mismas tablas. Cada equipo las interpreta a su manera, así que basta con que uno la haga mal para descuadrar el inventario de todos, y ningún otro se entera. Lo que hay que cambiar es eso: las tres reglas deben estar en la base de datos, que es el único punto por donde pasan todos los canales, y cada confirmación de pago debe guardarse completa o no guardarse, avisando cuando algo no se pudo aplicar en lugar de seguir callada. Además hay que cambiar cómo se prueba: hoy se prueba con un usuario a la vez y así todo funciona, pero el error solo aparece cuando varios clientes compran el mismo producto al mismo tiempo, y eso hay que probarlo antes de cada temporada alta.